

Worksheet No. - 04

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA-6B

Semester: 2nd

Date of Performance: 27.02.2026

Subject Name: Design and Analysis of Algorithm Lab **Subject Code:** 25CAP-612

Aim/Overview of the practical: To implement Dijkstra's Algorithm in C++ to find the shortest path from a single source vertex to all other vertices in a weighted graph using an adjacency matrix representation.

Objective: The objective of this program is to implement Dijkstra's algorithm in C++ to find the shortest path from a single source vertex to all other vertices in a graph. It aims to represent the graph using an adjacency matrix and calculate the minimum distances between nodes efficiently. The program also helps in understanding how shortest path algorithms work and how graphs are handled in programming.

Procedure/Algorithm/Code

Algorithm

1. Start and initialize all distances to infinity except the source vertex (set to 0).
2. Mark all vertices as unvisited.
3. Select the unvisited vertex with the minimum distance.
4. Mark the selected vertex as visited.
5. Update the distances of its adjacent vertices if a shorter path is found.
6. Repeat steps 3–5 until all vertices are visited.
7. Display the shortest distances from the source to all vertices.

Code

```
#include <iostream>

#include <climits>

using namespace std;

#define MAX 100

int minDistance(int dist[], bool visited[], int n) {

    int min = INT_MAX, min_index;

    for (int v = 0; v < n; v++) {

        if (!visited[v] && dist[v] <= min) {

            min = dist[v];

            min_index = v;

        }

    }

    return min_index;

}

void dijkstra(int graph[MAX][MAX], int n, int src) {

    int dist[MAX];

    bool visited[MAX];

    for (int i = 0; i < n; i++) {

        dist[i] = INT_MAX;

        visited[i] = false;

    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {

        int u = minDistance(dist, visited, n);
```

```
visited[u] = true;

for (int v = 0; v < n; v++) {

    if (!visited[v] && graph[u][v] &&

        dist[u] != INT_MAX &&

        dist[u] + graph[u][v] < dist[v]) {

        dist[v] = dist[u] + graph[u][v];

    }

}

}

cout << "\nVertex\tDistance from Source\n";

for (int i = 0; i < n; i++) {

    cout << i << "\t" << dist[i] << endl;

}

}

int main() {

    int n, src;

    int graph[MAX][MAX];

    cout << "Enter number of vertices: ";

    cin >> n;

    cout << "Enter the adjacency matrix:\n";

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < n; j++) {

            cin >> graph[i][j];

        }

    }
```

```
}  
  
cout << "Enter source vertex (0 to " << n-1 << "): ";  
  
cin >> src;  
  
dijkstra(graph, n, src);  
  
return 0;  
  
}
```

Output

```
Enter number of vertices: 4  
Enter the adjacency matrix:  
0 5 2 0  
5 0 1 3  
2 1 0 4  
0 3 4 0  
Enter source vertex (0 to 3): 2  
  
Vertex  Distance from Source  
0      2  
1      1  
2      0  
3      4  
  
=== Code Execution Successful ===
```

Learning outcomes (What I have learnt):

1. Understood the concept of shortest path in a weighted graph.
2. Learnt how Dijkstra's algorithm works step by step.
3. Implemented graph representation using an adjacency matrix in C++.